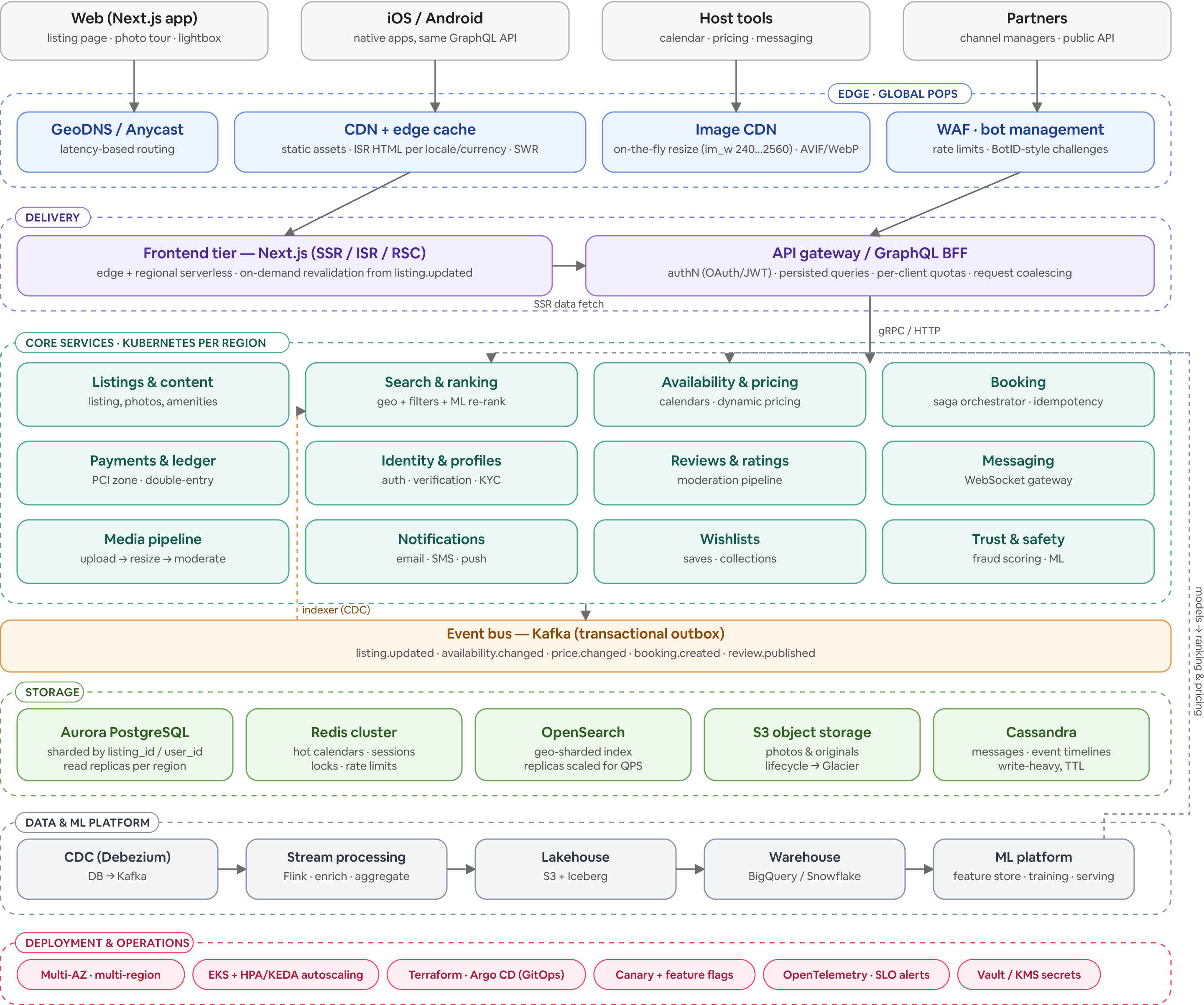


Vacation-rental marketplace — production architecture

Read paths (browse, search, listing pages) are cached and served globally; write paths (booking, payment) are strongly consistent and region-pinned. Events keep the two in sync.



Scaling strategy

Frontend

Listing pages SSR/ISR, cached at the edge with stale-while-revalidate; keys include locale + currency. listing.updated events trigger on-demand revalidation + purge. Photos via image CDN in width buckets; overlay state in the URL = cacheable links.

Backend

Stateless services autoscale on RPS/latency. CQRS: writes → Postgres; reads → Redis and search projections. Booking is a saga (hold → pay → confirm) with idempotency keys + transactional outbox. Circuit breakers and bulkheads per dependency.

Storage

Postgres sharded by listing_id / user_id, replicas per region; expand/contract migrations. Redis for hot calendars, locks, sessions. S3 media with lifecycle tiers. Cassandra for write-heavy messaging; ledger isolated in the PCI zone.

Search

OpenSearch geo-sharded (geohash routing), replicas sized for peak QPS. Near-real-time indexing via CDC → Kafka → indexer. Denormalised availability/price for filters; learning-to-rank re-ranking from the feature store; cache top destinations.

Deployment

Multi-AZ everywhere; browse/search active-active across regions, bookings pinned to a home region with async replication. GitOps canaries with auto rollback on SLO burn. DR targets: RPO ≤ 1 min, RTO ≤ 15 min.

This clone = the “Web” client: statically rendered Next.js page, CDN image loader, client state for booking + wishlist.